

OXVERSE ACADEMY

Web3 & Blockchain Development Curriculum

Professional Diploma · Cohort-based · Project-driven

DURATION

4 Months (16 Weeks)

TOTAL WEEKS

16

LEVEL

Beginner → Intermediate → Advanced → Industry Ready

PROJECTS

15+

CAPSTONE

1 Full-Stack Web3 dApp with Deployed Smart Contracts

PROGRAMME GOAL

Train students to become professional Web3 engineers who can design, write, test, secure, and ship production-grade smart contracts and dApps. Graduates leave with a deployed multi-contract portfolio, hands-on experience with the modern Ethereum and Solana toolchains, working knowledge of DeFi, NFTs, DAOs, and account abstraction, and the practical security instincts auditors and employers expect from a job-ready blockchain developer.

OVERVIEW

A rigorous 16-week professional diploma covering blockchain fundamentals, Ethereum and the EVM, Solidity from first principles to advanced patterns, the Hardhat and Foundry toolchains, modern dApp frontends with wagmi/viem, ERC20/ERC721/ERC1155 token standards, DeFi primitives (AMMs, lending, stablecoins), smart contract security and auditing, oracles and automation, Layer 2s and cross-chain bridges, account abstraction, an introduction to Solana and Anchor, DAOs and on-chain governance, and career preparation. Students build 15+ hands-on projects and finish with a fully deployed, tested, and documented full-stack Web3 capstone application.

No 82, Century Bus Stop, Ago Palace Way, Okota, Lagos · +234 913 869 1147

Course Outline

1. **Week 1:** Blockchain Foundations
2. **Week 2:** Ethereum Deep Dive
3. **Week 3:** Solidity Fundamentals
4. **Week 4:** Advanced Solidity
5. **Week 5:** Dev Tooling: Hardhat & Foundry
6. **Week 6:** Frontend for dApps
7. **Week 7:** Tokens: ERC20 & DeFi Primitives
8. **Week 8:** NFTs: ERC721 & ERC1155
9. **Week 9:** Smart Contract Security
10. **Week 10:** Oracles, Off-chain Data & Automation
11. **Week 11:** Layer 2s & Cross-chain
12. **Week 12:** Solana Intro
13. **Week 13:** DeFi Protocol Build
14. **Week 14:** DAOs & Governance
15. **Week 15:** Career & Ecosystem
16. **Week 16:** Capstone: Full-Stack dApp

Blockchain Foundations

Build a solid mental model of how blockchains work under the hood, from distributed ledgers and consensus to how wallets and keys actually secure user funds.

LEARNING OBJECTIVES

- Explain how distributed ledgers achieve consensus without a central authority
- Compare Proof of Work and Proof of Stake mechanics and tradeoffs
- Understand public/private key cryptography as it applies to crypto wallets
- Set up and safely use a self-custody wallet
- Read and interpret a blockchain explorer

DETAILED TOPIC BREAKDOWN

1.1 Blockchain 101

- What a distributed ledger is and why decentralization matters
- Blocks, hashes, and the Merkle tree structure
- How nodes propagate and validate transactions
- Proof of Work: mining, difficulty, and 51% attacks
- Proof of Stake: validators, staking, and slashing
- Byzantine Fault Tolerance and the double-spend problem
- Forks: soft forks vs hard forks with real examples
- Public, private, and consortium blockchains
- Blockchain trilemma: scalability, security, decentralization
- Reading whitepapers: Bitcoin and Ethereum whitepapers walkthrough

1.2 Bitcoin & Ethereum

- UTXO model vs account-based model explained with diagrams
- Bitcoin scripting basics and its limitations
- Ethereum as a world computer: state machine model
- Gas, gas price, gas limit, and base fee (EIP-1559)
- The Ethereum Virtual Machine (EVM) at a conceptual level
- Ether denominations: wei, gwei, ether
- Ethereum client types: execution vs consensus clients (Geth, Nethermind, Prysm)
- The Merge and Ethereum's move to Proof of Stake
- Mempools and transaction propagation
- MEV (Miner/Maximal Extractable Value) at a high level

1.3 Wallets & Keys

- Public-key cryptography and elliptic curve digital signatures (ECDSA)
- Private keys, public keys, and address derivation
- Seed phrases (BIP-39) and hierarchical deterministic wallets (BIP-32/44)
- Hot wallets vs cold wallets vs hardware wallets (Ledger, Trezor)
- Setting up and using MetaMask safely
- Custodial vs non-custodial wallets
- Common wallet scams: phishing, drainers, fake approvals
- Signing messages vs signing transactions
- Multi-signature wallets (Gnosis Safe) and their use cases
- Best practices for key management and seed phrase storage

HANDS-ON EXERCISES

- Manually construct a simplified block and compute its hash
- Simulate a 51% attack scenario on paper and explain the defense
- Create a MetaMask wallet and send a test transaction on a testnet

- Explore a transaction on Etherscan and identify all its fields
- Generate a seed phrase offline and derive addresses manually

ASSIGNMENTS

- Write a 1-page explainer comparing PoW and PoS for a non-technical audience
- Submit a screenshot walkthrough of your first testnet transaction with an explanation of each field

PROJECTS

- Personal Wallet Security Setup & Testnet Transaction Report

WEEKLY LEARNING OUTCOMES

- Understands how blockchains achieve trustless consensus
- Can explain gas, transactions, and the EVM at a conceptual level
- Can safely set up and operate a self-custody wallet
- Can read and interpret blockchain explorers

WEEKLY ASSESSMENT

Quiz on consensus mechanisms plus a graded wallet setup and transaction report

Ethereum Deep Dive

Go beneath the surface of Ethereum: understand EVM execution, account and transaction structure, and the wider Layer 2 ecosystem that scales it.

LEARNING OBJECTIVES

- Describe EVM storage, memory, and stack execution
- Differentiate externally owned accounts from contract accounts
- Understand transaction structure, nonces, and signatures
- Compare optimistic and zero-knowledge rollups
- Navigate the current Layer 2 ecosystem

DETAILED TOPIC BREAKDOWN

2.1 EVM

- EVM architecture: stack, memory, storage, calldata
- Opcodes and how bytecode executes step by step
- Gas costs per opcode and why some operations are expensive
- Storage slots and how state variables map to storage
- Memory vs storage vs calldata tradeoffs
- Contract creation and CREATE/CREATE2 opcodes
- Static calls, delegatecalls, and low-level calls
- Using EVM disassemblers and bytecode explorers
- Reading bytecode with tools like evm.codes

2.2 Accounts & Transactions

- Externally Owned Accounts (EOAs) vs contract accounts
- Transaction fields: nonce, gasLimit, gasPrice, to, value, data
- Legacy transactions vs EIP-1559 transactions
- Transaction signing and recovering signers (ecrecover)
- Nonce management and replay protection
- Transaction lifecycle from submission to finality
- Reorgs and probabilistic finality
- Reading raw transaction data with ethers.js and viem

2.3 Ecosystem

- Why Ethereum needs Layer 2 scaling
- Rollups overview: Optimism, Arbitrum, Base, zkSync, Starknet
- Sidechains vs rollups vs validiums
- Sequencers, data availability, and fraud/validity proofs
- Testnets: Sepolia, Holesky, and how to get testnet ETH
- Block explorers across chains: Etherscan, Arbiscan, Basescan
- RPC providers: Alchemy, Infura, QuickNode
- Ethereum roadmap: sharding, danksharding, proto-danksharding (EIP-4844)

HANDS-ON EXERCISES

- Trace a simple contract call opcode by opcode using evm.codes
- Decode a raw Ethereum transaction and identify each field
- Set up a free RPC endpoint with Alchemy and query chain data
- Compare gas costs of SSTORE vs SLOAD operations experimentally

ASSIGNMENTS

- Write a technical explainer of the transaction lifecycle from signing to finality

PROJECTS

- EVM Transaction Explorer Script using ethers.js or viem

- Submit an RPC-based script that fetches and prints the latest 5 blocks

WEEKLY LEARNING OUTCOMES

- Understands EVM execution at the opcode level
- Can construct and decode Ethereum transactions
- Can query chain data via RPC providers
- Understands the current Layer 2 landscape

WEEKLY ASSESSMENT

Technical quiz on EVM internals plus a graded RPC script submission

Solidity Fundamentals

Learn Solidity from the ground up: syntax, types, functions, and contract structure, with hands-on practice writing and deploying contracts in Remix.

LEARNING OBJECTIVES

- Write and compile basic Solidity contracts
- Use Solidity's type system, functions, and modifiers correctly
- Emit and understand the purpose of events
- Deploy and interact with contracts using Remix
- Debug simple contract errors and revert reasons

DETAILED TOPIC BREAKDOWN

3.1 Language Basics

Solidity file structure, pragma, and SPDX license identifiers Value types: uint, int, bool, address, bytes, enums

Reference types: arrays, structs, mappings Function visibility: public, private, internal, external

Function state mutability: view, pure, payable Custom modifiers and the require/revert/assert pattern

Custom errors vs string revert messages (gas comparison) Events, indexed parameters, and emitting logs

Constructors and contract initialization Global variables: msg.sender, msg.value, block.timestamp

3.2 Contracts

State variables and storage layout basics Inheritance: single and multiple inheritance, C3 linearization

Abstract contracts and interfaces Using libraries for reusable logic Function overriding and virtual/override keywords

Fallback and receive functions Contract-to-contract calls Error handling with try/catch for external calls

Solidity versioning and breaking changes across versions

3.3 Remix

Navigating the Remix IDE and file explorer Compiling contracts and reading compiler warnings

Deploying to the Remix VM, injected provider, and testnets Interacting with deployed contracts through the Remix UI

Reading transaction receipts and decoded logs in Remix Using the Remix debugger to step through failed transactions

Verifying contracts on Etherscan directly from Remix Installing and using Remix plugins (Solidity static analysis, Slither)

HANDS-ON EXERCISES

- Write a simple Counter contract with increment/decrement functions
- Build a Voting contract using mappings and structs
- Add custom errors and events to an existing contract
- Deploy a contract to Remix VM and trace a failing transaction with the debugger
- Refactor a contract to use inheritance and an interface

ASSIGNMENTS

PROJECTS

- Submit a fully commented Solidity contract implementing a simple to-do list with events
- Deploy a contract to Sepolia testnet and submit the verified Etherscan link
- On-Chain Voting DApp Contract deployed to a testnet

WEEKLY LEARNING OUTCOMES

- Can write, compile, and deploy Solidity contracts confidently
- Understands Solidity's type system and function modifiers
- Can use events for off-chain observability
- Comfortable debugging failed transactions in Remix

WEEKLY ASSESSMENT

Graded contract submission reviewed for correctness, gas awareness, and code style

Advanced Solidity

Move into production-grade Solidity: access control patterns, storage and gas optimization, and building on battle-tested OpenZeppelin contracts.

LEARNING OBJECTIVES

- Implement robust access control in smart contracts
- Optimize storage layout and gas usage
- Understand and apply upgradeable proxy patterns
- Use OpenZeppelin contracts for common standards
- Recognize common anti-patterns in contract design

DETAILED TOPIC BREAKDOWN

4.1 Patterns

- Ownable and role-based access control patterns
- OpenZeppelin's Ownable2Step and AccessControl contracts
- Multi-role permission systems for complex protocols
- Pausable contracts and circuit breaker patterns
- Upgradeable proxy patterns: Transparent, UUPS, Beacon
- Storage collision risks in upgradeable contracts
- Initializer functions vs constructors in upgradeable contracts
- Timelocks and multi-sig governed upgrades
- Diamond pattern (EIP-2535) overview for modular contracts

4.2 Storage & Gas

- How storage slots are packed and how to optimize struct layout
- Using uint128/uint64 packing to save gas
- Fixed-size arrays vs dynamic arrays gas tradeoffs
- Caching storage reads in memory during loops
- Short-circuiting and minimizing external calls
- Using immutable and constant to save deployment/runtime gas
- Gas profiling with Hardhat gas reporter and Foundry gas snapshots
- Batching operations to amortize fixed gas costs

4.3 OpenZeppelin

- Installing and importing OpenZeppelin Contracts library
- ERC20 implementation and extensions (Burnable, Pausable, Permit)
- ERC721 implementation and extensions (Enumerable, URIStorage)
- ERC1155 multi-token standard implementation
- ReentrancyGuard and SafeERC20 utilities
- Using OpenZeppelin Wizard to scaffold contracts
- OpenZeppelin Defender for operational security (monitoring, relayers)
- Auditing OpenZeppelin version changes and migration guides

HANDS-ON EXERCISES

- Refactor a contract to use OpenZeppelin AccessControl with two roles
- Optimize a struct's storage layout and measure gas savings
- Build an upgradeable contract using the UUPS proxy pattern
- Add ReentrancyGuard to a vulnerable withdraw function

ASSIGNMENTS

PROJECTS

- Submit an upgradeable token contract with role-based minting permissions
- Write a short report comparing gas costs before and after an optimization pass
- Upgradeable ERC20 Token with Role-Based Access Control

WEEKLY LEARNING OUTCOMES

- Can implement production-grade access control
- Understands storage packing and gas optimization techniques
- Can build and reason about upgradeable contracts
- Comfortable extending OpenZeppelin standard contracts

WEEKLY ASSESSMENT

Graded contract review focused on security, gas efficiency, and correct use of OpenZeppelin patterns

Dev Tooling: Hardhat & Foundry

Master the two dominant Ethereum development frameworks, and build a rigorous testing discipline with unit, fuzz, and invariant tests.

LEARNING OBJECTIVES

- Set up and configure Hardhat and Foundry projects
- Write deployment and interaction scripts
- Write comprehensive unit and fuzz tests
- Use invariant testing to catch edge-case bugs
- Measure and interpret test coverage reports

DETAILED TOPIC BREAKDOWN

5.1 Hardhat

- Setting up a Hardhat project with TypeScript
- hardhat.config.ts: networks, compilers, plugins
- Writing deployment scripts with hardhat-deploy or ignition
- Hardhat Network: forking mainnet for local testing
- Using hardhat-verify to verify contracts on Etherscan
- Console.log debugging inside Solidity with hardhat/console.sol
- Task creation and custom Hardhat tasks
- Managing multiple networks and environment variables safely

5.2 Foundry

- Installing Foundry: forge, cast, anvil, chisel
- Writing tests in Solidity with forge-std
- Using cheatcodes: vm.prank, vm.deal, vm.warp, vm.expectRevert
- Local chain simulation with Anvil
- Using cast to query chain state and send transactions from CLI
- Forking live networks for integration testing
- forge script for deployments
- Foundry.toml configuration and remappings

5.3 Testing

- Unit testing philosophy for smart contracts
- Fuzz testing with random inputs in Foundry
- Invariant testing: defining protocol invariants that must always hold
- Property-based testing patterns
- Test coverage with forge coverage and solidity-coverage
- Mocking external contracts and oracles in tests
- Continuous integration for smart contracts with GitHub Actions
- Static analysis integration into CI (Slither, Mythril)

HANDS-ON EXERCISES

- Migrate a Remix contract into a Hardhat project with tests
- Write a Foundry fuzz test for an ERC20 transfer function
- Write an invariant test asserting total supply never changes unexpectedly
- Set up a GitHub Actions workflow that runs Foundry tests on push

ASSIGNMENTS

- Submit a Hardhat or Foundry repository with 90%+ test coverage on a token contract

PROJECTS

- Fully Tested Vault Contract with CI Pipeline

- Write and submit an invariant test suite for a simple vault contract

WEEKLY LEARNING OUTCOMES

- Comfortable working in both Hardhat and Foundry
- Can write unit, fuzz, and invariant tests
- Can set up CI pipelines for smart contract projects
- Understands how to measure and improve test coverage

WEEKLY ASSESSMENT

Graded repository review of test suite quality, coverage, and CI configuration

Frontend for dApps

Bring smart contracts to life with a modern React frontend, wallet connection flows, and real-time contract interaction using wagmi and viem.

LEARNING OBJECTIVES

- Refresh core React and Tailwind skills for dApp UIs
- Connect wallets using modern libraries
- Read from and write to smart contracts from the frontend
- Listen to on-chain events in real time
- Handle transaction states gracefully in the UI

DETAILED TOPIC BREAKDOWN

6.1 React Refresh

- Functional components, hooks, and state management refresher
- React Router for multi-page dApp navigation
- Tailwind CSS setup and utility-first styling
- Component composition patterns for dApp UIs
- Managing loading, error, and success states in the UI
- Environment variables and config management with Vite

6.2 Wallet Connection

- wagmi hooks: useAccount, useConnect, useDisconnect
- viem clients: public client vs wallet client
- RainbowKit setup for polished wallet connection UI
- ConnectKit as an alternative wallet connection library
- WalletConnect v2 integration for mobile wallets
- Handling network switching and chain mismatches
- Persisting connection state across page reloads
- Detecting and handling unsupported wallets/browsers

6.3 Contract Interaction

- Importing and typing contract ABIs in a frontend project
- useReadContract and useWriteContract hooks with wagmi
- Simulating transactions before sending (useSimulateContract)
- Waiting for transaction confirmations and handling receipts
- Listening to contract events with watchContractEvent
- Formatting and parsing units (formatEther, parseUnits)
- Handling and surfacing revert reasons to users
- Optimistic UI updates and transaction toasts

HANDS-ON EXERCISES

- Build a wallet-connect button with RainbowKit and display the connected address
- Build a UI that reads and displays a contract's state variable
- Build a form that calls a contract write function and shows transaction status
- Subscribe to a contract event and display live updates in the UI

ASSIGNMENTS

- Submit a working dApp frontend connected to your Week 3 voting contract
- Add transaction status UI (pending, success, failed) to a write flow

PROJECTS

- Voting DApp Frontend with Live Wallet Connection and Contract Interaction

WEEKLY LEARNING OUTCOMES

- Can build production-quality dApp frontends
- Can connect wallets and manage chain state in React
- Can read from and write to contracts using wagmi/viem
- Can handle transaction lifecycle states gracefully in the UI

WEEKLY ASSESSMENT

Live demo and code review of the working dApp frontend

Tokens: ERC20 & DeFi Primitives

Understand fungible tokens deeply and explore the core building blocks of DeFi: automated market makers, lending protocols, and stablecoins.

LEARNING OBJECTIVES

- Implement and extend ERC20 tokens correctly
- Design token vesting and distribution mechanisms
- Explain how AMMs, lending protocols, and stablecoins work
- Understand Uniswap's architecture at a conceptual level
- Interact with a live AMM contract from code

DETAILED TOPIC BREAKDOWN

7.1 ERC20

- ERC20 standard functions and events in depth
- Approve/transferFrom pattern and allowance race conditions
- ERC20Permit (EIP-2612) for gasless approvals
- Minting strategies: fixed supply, capped supply, inflationary
- Token vesting contracts: cliffs, linear vesting, revocability
- Airdrop mechanics and Merkle-tree-based claims
- Token burning and deflationary mechanisms
- Common ERC20 pitfalls: non-standard tokens, fee-on-transfer tokens

7.2 DeFi Basics

- Automated Market Makers (AMMs) and the constant product formula
- Liquidity pools, LP tokens, and impermanent loss
- Lending and borrowing protocols: collateralization, liquidation
- Interest rate models (Aave, Compound style)
- Stablecoins: fiat-collateralized, crypto-collateralized, algorithmic
- Flash loans and their legitimate and exploit use cases
- Yield farming and liquidity mining incentives
- Total Value Locked (TVL) and DeFi metrics

7.3 Uniswap Intro

- Uniswap v2 architecture: factory, pair, router contracts
- Uniswap v3 concentrated liquidity model
- Price impact and slippage tolerance
- Reading pool reserves and calculating swap output
- Interacting with Uniswap contracts via ethers.js/viem
- Fee tiers in Uniswap v3 and their tradeoffs
- Using the Uniswap SDK for quote calculations

HANDS-ON EXERCISES

- Build an ERC20 token with a linear vesting schedule
- Implement a Merkle-tree airdrop claim contract
- Calculate swap output manually using the constant product formula
- Query live Uniswap pool reserves and compute the current price

ASSIGNMENTS

- Submit a vesting token contract with tests covering cliff and linear release

PROJECTS

- Token Vesting & Airdrop System with Merkle Proof Claims

- Write a report explaining impermanent loss with a worked numeric example

WEEKLY LEARNING OUTCOMES

- Can implement advanced ERC20 token mechanics
- Understands core DeFi primitives conceptually and mathematically
- Can interact with live AMM contracts programmatically
- Understands risk factors like impermanent loss and liquidation

WEEKLY ASSESSMENT

Graded submission of the vesting/airdrop contract plus the impermanent loss report

NFTs: ERC721 & ERC1155

Master non-fungible and multi-token standards, decentralized storage for metadata, and how NFT marketplaces work under the hood.

LEARNING OBJECTIVES

- Implement ERC721 and ERC1155 contracts with metadata
- Store and pin NFT assets on decentralized storage
- Implement royalties and marketplace-compatible listings
- Build a minting dApp with a live frontend
- Understand marketplace integration standards

DETAILED TOPIC BREAKDOWN

8.1 Standards

- ERC721 standard: ownership, transfers, approvals
- ERC721Enumerable and ERC721URIStorage extensions
- ERC1155 multi-token standard for fungible and non-fungible items
- Batch operations in ERC1155 (batch transfer, batch balance)
- Metadata JSON schema for NFTs (name, description, image, attributes)
- On-chain vs off-chain metadata tradeoffs
- Soulbound tokens (non-transferable NFTs) and their use cases
- Lazy minting patterns to reduce upfront gas costs

8.2 IPFS & Storage

- How IPFS content addressing works (CIDs)
- Pinning services: Pinata, nft.storage, web3.storage
- Uploading images and metadata JSON to IPFS
- Arweave as a permanent storage alternative
- Generating and hosting token metadata dynamically
- IPFS gateways and their reliability tradeoffs
- Storing large media assets efficiently

8.3 Marketplaces

- EIP-2981 royalty standard implementation
- Listing NFTs for sale: fixed price vs auction models
- Escrow-based marketplace contract design
- Integrating with OpenSea (Seaport protocol overview)
- Marketplace fee structures and creator royalties enforcement challenges
- Signature-based listings (off-chain order, on-chain settlement)
- Handling collection verification and metadata refresh

HANDS-ON EXERCISES

- Deploy an ERC721 contract and mint an NFT with IPFS-hosted metadata
- Implement EIP-2981 royalties in an ERC721 contract
- Build an ERC1155 contract with 3 different token types
- Build a simple escrow-based NFT listing contract

ASSIGNMENTS

- Submit a deployed NFT collection with metadata pinned on IPFS via Pinata

PROJECTS

- NFT Minting DApp with IPFS Metadata and On-Chain Royalties

- Write documentation explaining your royalty and marketplace design choices

WEEKLY LEARNING OUTCOMES

- Can implement and deploy ERC721/ERC1155 contracts
- Can store and pin NFT assets on decentralized storage
- Understands royalty standards and marketplace mechanics
- Can build a complete minting dApp end to end

WEEKLY ASSESSMENT

Graded review of the deployed NFT collection, metadata setup, and royalty implementation

Smart Contract Security

Develop an attacker's mindset: study real exploit patterns, learn to use static analysis tools, and internalize secure coding best practices.

LEARNING OBJECTIVES

- Identify and explain the most common smart contract vulnerabilities
- Use automated tools to detect security issues
- Apply secure design patterns to prevent exploits
- Analyze real-world hack post-mortems
- Perform a basic manual security review of a contract

DETAILED TOPIC BREAKDOWN

9.1 Common Vulns

- Reentrancy attacks: classic and cross-function reentrancy
- Integer overflow/underflow (pre and post Solidity 0.8)
- Access control failures and unprotected functions
- Oracle manipulation and price feed exploits
- Front-running and sandwich attacks
- Denial of service via unexpected reverts or gas limits
- Delegatecall injection and storage collision attacks
- Signature replay attacks and missing nonce checks
- Rounding errors and precision loss exploits
- Case studies: The DAO hack, Cream Finance, Euler Finance

9.2 Auditing Basics

- Manual code review methodology and checklists
- Using Slither for automated static analysis
- Using Mythril for symbolic execution analysis
- Interpreting and triaging static analysis findings
- Writing exploit proof-of-concept tests in Foundry
- Reading public audit reports (Trail of Bits, OpenZeppelin, Consensys Diligence)
- Introduction to formal verification concepts
- Bug bounty platforms: Immunefi and how disclosures work

9.3 Best Practices

- Checks-Effects-Interactions pattern in depth
- Pull-over-push payment pattern
- Using ReentrancyGuard correctly
- Timelocks for privileged operations
- Circuit breakers and emergency pause mechanisms
- Rate limiting and withdrawal caps
- Secure randomness with Chainlink VRF instead of blockhash
- Writing a security checklist before mainnet deployment

HANDS-ON EXERCISES

- Exploit a deliberately vulnerable reentrancy contract in a sandbox
- Run Slither on a contract and fix all flagged high-severity issues
- Write a Foundry PoC test reproducing a known historical exploit
- Perform a manual review of a peer's contract using a checklist

ASSIGNMENTS

- Submit a security audit report of a provided vulnerable contract with fixes

PROJECTS

- Mini Security Audit Report on a Vulnerable Sample Protocol

- Write a case study analysis of a real DeFi hack including root cause and fix

WEEKLY LEARNING OUTCOMES

- Can identify common smart contract vulnerabilities
- Can use Slither and Mythril for automated analysis
- Understands and applies secure design patterns
- Can write a structured security audit report

WEEKLY ASSESSMENT

Graded audit report evaluated on vulnerability coverage, accuracy, and remediation quality

Oracles, Off-chain Data & Automation

Learn how smart contracts safely consume off-chain data and automate execution using Chainlink and how The Graph indexes on-chain events.

LEARNING OBJECTIVES

- Integrate Chainlink Data Feeds for reliable price data
- Use Chainlink VRF for verifiable randomness
- Automate contract functions with Chainlink Automation or Gelato
- Build and query a subgraph with The Graph
- Understand oracle security tradeoffs

DETAILED TOPIC BREAKDOWN

10.1 Oracles

- Why blockchains cannot natively access off-chain data
- Chainlink Data Feeds architecture and decentralized aggregation
- Integrating an AggregatorV3Interface price feed into a contract
- Chainlink VRF for verifiable random number generation
- Handling VRF requests and fulfillments correctly
- Oracle manipulation risks and mitigation (TWAP, multiple sources)
- Alternative oracle providers: Pyth Network, API3
- Push vs pull oracle models

10.2 Automation

- Why smart contracts cannot self-trigger and need keepers
- Chainlink Automation (Keepers) upkeep registration and funding
- Writing checkUpkeep and performUpkeep functions
- Gelato Network as an alternative automation provider
- Gas considerations for automated tasks
- Use cases: automated liquidations, rebasing, reward distribution
- Monitoring and alerting for automation failures

10.3 Indexing

- Why indexing on-chain data is necessary for performant frontends
- The Graph architecture: subgraphs, indexers, GraphQL
- Defining a subgraph.yaml and schema.graphql
- Writing AssemblyScript mappings for event handling
- Deploying a subgraph to The Graph's hosted service or decentralized network
- Querying a subgraph with GraphQL from a React frontend
- Alternatives: Dune Analytics, Ponder, custom indexers

HANDS-ON EXERCISES

- Integrate a Chainlink ETH/USD price feed into a contract
- Request and fulfill a random number using Chainlink VRF on testnet
- Register a Chainlink Automation upkeep for a scheduled function
- Build and deploy a subgraph indexing Transfer events for an ERC20 token

ASSIGNMENTS

- Submit a contract using a live price feed with a report on oracle risk mitigation

PROJECTS

- Price-Feed-Powered Contract with Automated Upkeep and a Custom Subgraph

- Submit a working subgraph with example GraphQL queries and results

WEEKLY LEARNING OUTCOMES

- Can integrate Chainlink oracles into smart contracts
- Can automate on-chain execution with Automation or Gelato
- Can build and query a subgraph with The Graph
- Understands oracle security tradeoffs

WEEKLY ASSESSMENT

Graded review of the oracle integration, automation setup, and subgraph query results

Layer 2s & Cross-chain

Understand how Ethereum scales through rollups, how assets move across chains via bridges, and how account abstraction is changing wallet UX.

LEARNING OBJECTIVES

- Differentiate optimistic and zero-knowledge rollups
- Deploy and interact with contracts across multiple L2 networks
- Understand bridge architectures and their security tradeoffs
- Explain account abstraction (ERC-4337) and its benefits
- Evaluate risks of cross-chain infrastructure

DETAILED TOPIC BREAKDOWN

11.1 Rollups

- Optimistic rollups: fraud proofs and challenge periods
- Zero-knowledge rollups: validity proofs and zk-SNARKs/STARKs basics
- Base, Arbitrum, and Optimism architecture comparison
- zkSync Era and Starknet architecture overview
- Data availability and EIP-4844 blob transactions
- Sequencer centralization concerns and decentralization roadmaps
- Deploying the same contract across multiple L2s
- Gas cost differences between L1 and L2 deployments

11.2 Bridges

- Native/canonical bridges vs third-party bridges
- Lock-and-mint vs burn-and-mint bridge models
- Liquidity network bridges (Across, Hop) vs message-passing bridges
- Bridge security failures and historical exploits (Ronin, Wormhole)
- LayerZero and cross-chain messaging protocols
- Trust assumptions: multisig-secured vs light-client-secured bridges
- Evaluating bridge risk before integrating into a dApp

11.3 Account Abstraction

- Limitations of EOAs that account abstraction solves
- ERC-4337 architecture: UserOperations, bundlers, EntryPoint
- Smart contract wallets: Safe, Argent, Biconomy
- Paymasters and gasless (sponsored) transactions
- Social recovery and multi-factor smart wallet security
- Session keys for improved dApp UX
- Building a simple smart account integration with a paymaster

HANDS-ON EXERCISES

- Deploy the same contract to two different L2 testnets and compare gas costs
- Trace a cross-chain asset transfer using a bridge explorer
- Set up a smart contract wallet using Safe on testnet
- Simulate a gasless transaction flow using a paymaster on testnet

ASSIGNMENTS

- Write a comparison report of 3 rollups covering architecture, fees, and ecosystem

PROJECTS

- Multi-Chain Contract Deployment with a Gasless Smart Account Demo

- Submit a working ERC-4337 smart account demo with a sponsored transaction

WEEKLY LEARNING OUTCOMES

- Understands rollup architectures and their tradeoffs
- Can deploy and manage contracts across multiple L2s
- Understands bridge security models and risks
- Can build a working account abstraction demo

WEEKLY ASSESSMENT

Graded review of the multi-chain deployment report and the smart account demo

Solana Intro

Step outside the EVM to learn Solana's distinct account model, write your first Rust-based Anchor program, and connect it to a frontend.

LEARNING OBJECTIVES

- Explain Solana's account model and how it differs from Ethereum
- Write basic Rust for Solana program development
- Build, test, and deploy a program with the Anchor framework
- Create and manage SPL tokens
- Connect a Solana program to a React frontend

DETAILED TOPIC BREAKDOWN

12.1 Solana Basics

- Solana's account model: everything is an account
- Programs vs data accounts and rent-exemption
- Program Derived Addresses (PDAs) and their use cases
- Solana's parallel execution model (Sealevel) and why it's fast
- Transactions, instructions, and signers on Solana
- SPL Token program and the token account model
- Solana CLI: keypairs, airdrops, and cluster configuration
- Comparing Solana's fee model to Ethereum's gas model

12.2 Anchor Framework

- Rust basics needed for Anchor: structs, traits, Result types
- Anchor project structure and workspace setup
- Defining accounts with `#[account]` and account constraints
- Writing instruction handlers in an Anchor program
- Anchor's IDL (Interface Definition Language) generation
- Testing Anchor programs with TypeScript and Mocha
- Local validator testing with `solana-test-validator`
- Deploying an Anchor program to Devnet

12.3 Solana dApps

- Setting up `@solana/web3.js` in a React project
- Solana Wallet Adapter for Phantom, Solflare, and other wallets
- Fetching account data and deserializing with the Anchor IDL
- Sending instructions from the frontend using Anchor's TypeScript client
- Displaying SPL token balances in the UI
- Subscribing to account changes with WebSocket connections
- Differences in UX considerations between EVM and Solana dApps

HANDS-ON EXERCISES

- Set up a Solana CLI wallet and request a Devnet airdrop
- Write and deploy a simple Anchor counter program to Devnet
- Create a custom SPL token and mint it to a wallet
- Build a minimal React page that connects a Phantom wallet and reads program state

ASSIGNMENTS

- Submit a deployed Anchor program with a passing TypeScript test suite

PROJECTS

- Solana Counter/Voting Program with Anchor and a Connected Frontend

- Submit a frontend that reads and displays data from your deployed program

WEEKLY LEARNING OUTCOMES

- Understands Solana's account model and how it differs from the EVM
- Can write, test, and deploy Anchor programs
- Can create and manage SPL tokens
- Can connect a Solana program to a working frontend

WEEKLY ASSESSMENT

Graded review of the deployed Anchor program, tests, and connected frontend

DeFi Protocol Build

Apply everything learned so far to design and build a complete DeFi protocol from scratch, from token economics to a working frontend.

LEARNING OBJECTIVES

- Design a simple lending or DEX protocol from first principles
- Model token economics and incentive alignment
- Implement the protocol with thorough tests
- Connect the protocol to a functional frontend
- Present and defend design decisions

DETAILED TOPIC BREAKDOWN

13.1 Design

- Choosing between a simple lending pool or a constant-product DEX
- Defining protocol invariants and safety properties upfront
- Token model design: governance, utility, or fee-sharing tokens
- Collateralization ratios and liquidation thresholds for lending
- Fee structures and protocol revenue design
- Writing a lightweight protocol specification document
- Identifying attack surfaces before writing code

13.2 Implementation

- Structuring a multi-contract protocol repository
- Implementing core swap or lend/borrow logic in Solidity
- Writing comprehensive unit, fuzz, and invariant tests
- Integrating price oracles where needed
- Adding access control and emergency pause mechanisms
- Building a frontend for deposits, swaps, or borrowing flows
- Deploying the full protocol to a public testnet
- Writing user-facing and developer documentation

HANDS-ON EXERCISES

- Write a one-page protocol specification before writing any code
- Implement and test the core swap or lend function in isolation
- Add an invariant test proving the protocol cannot lose solvency
- Build a minimal frontend flow for the core user action (swap, deposit, borrow)

ASSIGNMENTS

- Submit the protocol specification document for peer and instructor review
- Submit the deployed protocol contracts with a full test suite

PROJECTS

- Simple DEX or Lending Protocol with Frontend, Deployed to Testnet

WEEKLY LEARNING OUTCOMES

- Can design a DeFi protocol from first principles
- Can implement and thoroughly test protocol logic
- Can connect a protocol to a working frontend
- Can articulate and defend design tradeoffs

WEEKLY ASSESSMENT

Project defense presentation and graded code review of the deployed protocol

DAOs & Governance

Explore how decentralized organizations coordinate and vote on-chain, and use industry-standard tooling to launch and manage a DAO.

LEARNING OBJECTIVES

- Compare different DAO voting models
- Deploy a governance-enabled token and governor contract
- Set up off-chain signaling with Snapshot
- Manage on-chain execution of passed proposals with a timelock
- Evaluate DAO governance risks and attack vectors

DETAILED TOPIC BREAKDOWN

14.1 DAO Basics

- What a DAO is and how it differs from a traditional organization
- Token-weighted voting and its plutocracy risks
- Quadratic voting and quadratic funding concepts
- Delegation and vote delegation mechanics
- Proposal lifecycle: creation, discussion, voting, execution
- Treasury management and multisig-controlled funds
- Governance attack vectors: flash loan governance attacks, vote buying
- Real-world DAO case studies: Uniswap, MakerDAO, Compound

14.2 Tooling

- Snapshot for gasless off-chain voting signaling
- Setting up a Snapshot space and creating proposals
- OpenZeppelin Governor contract architecture
- Governor extensions: GovernorVotes, GovernorTimelockControl
- TimelockController for delayed, transparent execution
- Tally as a governance dashboard and proposal interface
- ERC20Votes and checkpoint-based voting power
- Multisig integration (Gnosis Safe) with DAO treasuries

HANDS-ON EXERCISES

- Deploy an ERC20Votes token and delegate voting power to yourself
- Deploy an OpenZeppelin Governor contract with a timelock
- Create and vote on a proposal end to end on a testnet
- Set up a Snapshot space and create an off-chain proposal

ASSIGNMENTS

- Submit a deployed governance system (token + governor + timelock) with a passed test proposal
- Write a short report analyzing a real DAO governance attack and its root cause

PROJECTS

- On-Chain DAO Governance System with Timelocked Execution

WEEKLY LEARNING OUTCOMES

- Understands different DAO voting models and their tradeoffs
- Can deploy a full governance stack with OpenZeppelin Governor
- Can set up off-chain signaling with Snapshot

- Can identify common governance attack vectors

WEEKLY ASSESSMENT

Graded review of the deployed governance system and the governance attack case study

Career & Ecosystem

Package your work into a compelling professional portfolio, engage with the Web3 community, and prepare for technical interviews.

LEARNING OBJECTIVES

- Build a professional Web3 developer portfolio
- Document projects with clear case studies
- Identify hackathons, grants, and open-source opportunities
- Practice Solidity and system design interview questions
- Build a personal brand within the Web3 community

DETAILED TOPIC BREAKDOWN

15.1 Portfolio

- Structuring a GitHub profile for maximum recruiter impact
- Writing clear README files with architecture diagrams
- Showcasing deployed contract addresses and verified source code
- Building a personal portfolio site listing all capstone projects
- Writing detailed case studies for 3-5 flagship projects
- Recording short demo videos of working dApps
- Highlighting security practices and test coverage in project write-ups

15.2 Community

- Finding and participating in Web3 hackathons (ETHGlobal, Solana hackathons)
- Applying for ecosystem grants (Gitcoin, protocol-specific grant programs)
- Contributing to open-source Web3 repositories
- Networking on Farcaster, X, and Discord developer communities
- Following and engaging with core protocol teams and researchers
- Building in public: sharing progress and learnings online

15.3 Interviews

- Common Solidity coding interview questions and how to approach them
- System design interviews for blockchain protocols
- Whiteboarding a token or DeFi protocol architecture live
- Explaining security tradeoffs clearly under interview pressure
- Mock technical interview practice with peer review
- Negotiating offers and understanding Web3 compensation structures (token vesting, equity)

HANDS-ON EXERCISES

- Rewrite your GitHub README for your best project with a clear architecture diagram
- Record a 2-minute demo video for your NFT or DeFi project
- Complete 5 Solidity interview coding challenges under time pressure
- Do a mock system design interview with a peer

ASSIGNMENTS

- Submit a completed portfolio site linking all major projects with case studies

PROJECTS

- Professional Web3 Developer Portfolio Site

- Submit a written application to one real hackathon or grant program

WEEKLY LEARNING OUTCOMES

- Has a polished, recruiter-ready Web3 portfolio
- Understands where to find hackathons, grants, and OSS opportunities
- Comfortable with Solidity and system design interview formats
- Has taken concrete steps toward community visibility

WEEKLY ASSESSMENT

Portfolio review and mock interview scored on technical depth and communication clarity

Capstone: Full-Stack dApp

Design, build, secure, and ship one complete full-stack Web3 application from specification through deployed contracts to a polished, documented frontend.

LEARNING OBJECTIVES

- Write a clear technical specification for an original dApp
- Implement and thoroughly test the smart contract layer
- Deploy contracts to a public testnet or mainnet
- Build a polished, fully functional frontend
- Perform a self-audit and produce professional documentation

DETAILED TOPIC BREAKDOWN

16.1 Spec

- Choosing a capstone problem with real user value
- Writing a product and technical specification document
- Defining smart contract architecture and data flow diagrams
- Planning frontend screens, states, and wallet flows
- Scoping an MVP feature set achievable within the timeline
- Getting instructor and peer feedback on the spec before building

16.2 Build

- Implementing all smart contracts with full unit and fuzz test coverage
- Deploying contracts to a public testnet (and mainnet if applicable)
- Verifying contracts on a block explorer
- Building the complete frontend with wallet connection and all core flows
- Integrating event listening and real-time state updates
- Handling edge cases: failed transactions, network switching, empty states
- Polishing UI/UX for a professional, demo-ready product
- Setting up a live deployment (Vercel/Netlify) for the frontend

16.3 Audit & Docs

- Running Slither and manual review as a self-audit pass
- Fixing all identified high and medium severity issues
- Writing a README with setup instructions and architecture overview
- Documenting all deployed contract addresses and verification links
- Recording a final demo video walking through the dApp
- Presenting the capstone to instructors and peers with a live Q&A

HANDS-ON EXERCISES

- Peer-review a classmate's capstone specification before building begins
- Run a self-audit checklist against your own contracts before deployment
- Do a dry-run demo of your dApp and collect feedback for polish

ASSIGNMENTS

- Submit the capstone technical specification for approval
- Submit the final deployed dApp with full documentation and demo video

PROJECTS

- Full-Stack Web3 dApp with Deployed Smart Contracts

WEEKLY LEARNING OUTCOMES

- Can independently scope and specify a full-stack Web3 product
- Can implement, test, and deploy production-quality smart contracts
- Can build a polished, fully functional dApp frontend
- Can self-audit and professionally document a shipped project

WEEKLY ASSESSMENT

Final capstone presentation and live demo graded against a comprehensive rubric covering architecture, security, and polish